Light

FDE Cycle & Systems Software

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. The FDE cycle
2. Operating systems
3. Utility and robust software
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **FDE Cycle and Systems Software (1CP2/01 Topic 3: Computers)**. This topic covers how the CPU executes instructions through the fetch-decode-execute cycle, the functions of operating systems, the purpose of utility software, and principles of robust software including audit trails and code reviews.
Exam questions typically ask you to **describe** the stages of the FDE cycle, **explain** the functions of operating systems, **identify** utility software purposes, and **explain** how audit trails improve accountability. The 2024 Paper 1 included questions on compilers, file systems, and audit trails. Expect questions worth 2—6 marks requiring linked descriptions.

Why this topic matters

Understanding how computers execute instructions and manage resources is fundamental to computer science:

- **FDE cycle:** Every program you write is executed through this cycle — understanding it helps you appreciate how code becomes action
- **Operating systems:** The OS manages all hardware and software resources — knowing its functions explains why computers work the way they do

- **Utility software:** These tools maintain system performance and security — essential knowledge for troubleshooting and maintenance
- **Robust software:** Audit trails and code reviews are industry-standard practices for accountability and quality — tested in the 2024 exam
- **Exam relevance:** This topic typically appears in Paper 1 Question 4, worth 10—15 marks across multiple sub-questions

What you must know

This checklist covers the key exam-visible concepts from Edexcel 1CP2 Topic 3: Computers. You should be able to explain each point and apply it to exam scenarios.

- **Describe** the three stages of the fetch-decode-execute cycle and state which CPU components are involved at each stage
- **Explain** the role of the Program Counter, MAR, MDR and Control Unit during the FDE cycle
- **State** the need for operating systems and **describe** their four main functions: file management, process management, peripheral management, user management
- **Identify** the purpose of utility software including file repair, backup, compression, defragmentation, and anti-malware
- **Explain** how audit trails improve accountability and how code reviews identify vulnerabilities
- **Describe** the characteristics of a compiler and how it translates high-level code

> **Specification link:** This topic covers Edexcel 1CP2 Paper 1 Topic 3: Computers — specifically the FDE cycle (hardware components), operating system functions, utility software purposes, and robust software practices (audit trails, code reviews, vulnerability identification).

Toolkit

These are the key patterns, definitions and relationships you need to apply in exam questions.

FDE cycle stage summary

Stage	What happens	Components involved
Fetch	The next instruction is retrieved from memory and placed in the MDR	PC, MAR, MDR, Address bus, Data bus, RAM
Decode	The Control Unit interprets the instruction to determine what operation is required	CU, MDR
Execute	The instruction is carried out — this may involve calculations, data movement, or control flow changes	ALU, Accumulator, CU, Registers

Operating system functions

Function	What it does (exam wording)	Examples
File management	Organises files and folders; controls read/write access; maintains directory structure	Creating folders, copying files, setting permissions, tracking file locations
Process management	Allocates CPU time to running programs; handles multitasking; manages process priorities	Running multiple apps simultaneously, ending unresponsive programs
Peripheral management	Controls communication between the CPU and peripheral devices; manages device drivers	Sending documents to printer, receiving keyboard input, recognising USB devices
User management	Controls user accounts, authentication, and access rights; maintains security	Login screens, user profiles, admin vs standard user permissions

Utility software summary

Utility	Purpose (exam wording)
File repair	Detects and fixes errors in file systems; recovers corrupted files
Backup	Creates copies of data to protect against loss; allows restoration after failure
Compression	Reduces file sizes to save storage space or speed up transmission
Defragmentation	Reorganises fragmented files on a hard disk so they are stored contiguously, improving access speed
Anti-malware	Detects, quarantines and removes malicious software such as viruses and ransomware

Robust software concepts

Audit trail: A record that tracks who made changes to data, when changes were made, and what was changed. Improves accountability because actions can be traced back to individuals.
Code review: The process of having other programmers examine source code to identify bugs, vulnerabilities, and areas for improvement before deployment.
Vulnerability identification: The process of finding security weaknesses in software that could be exploited by attackers.

1. The fetch-decode-execute cycle

The **fetch-decode-execute (FDE) cycle** is the fundamental process by which a CPU executes instructions. Every program, from a simple calculator to a complex game, runs

by repeatedly cycling through these three stages. The cycle continues until the program ends or the computer is shut down.

1.1 Overview of the cycle

The FDE cycle is a continuous loop that the CPU performs billions of times per second. Each cycle processes one instruction from a program stored in memory.

The three stages:

- **Fetch:** Get the next instruction from memory
- **Decode:** Work out what the instruction means
- **Execute:** Carry out the instruction

The speed at which this cycle repeats is determined by the **clock speed**. A 3 GHz processor can perform approximately 3 billion clock cycles per second, though complex instructions may take multiple cycles to complete.

1.2 The fetch stage

During the fetch stage, the CPU retrieves the next instruction from main memory (RAM). This involves several registers working together:

1. The **Program Counter (PC)** holds the memory address of the next instruction
2. This address is copied from the PC to the **Memory Address Register (MAR)**
3. The address in the MAR is sent to memory via the **address bus**
4. The instruction at that address is fetched and placed in the **Memory Data Register (MDR)** via the **data bus**
5. The PC is incremented to point to the next instruction in sequence

> **Exam tip:** When describing the fetch stage, always mention the PC → MAR → address bus → memory → data bus → MDR sequence. This shows you understand the flow of data between components.

1.3 The decode stage

During the decode stage, the **Control Unit (CU)** interprets the instruction that was fetched:

1. The instruction is passed from the MDR to the **Control Unit**
2. The CU **decodes** the instruction to determine what operation is needed
3. The CU identifies what data is required and where it is located
4. The CU prepares the appropriate signals to send to other components

The CU works out what action the instruction requires (e.g., add two numbers, load data from memory, jump to a different instruction) and coordinates the other components to carry it out.

1.4 The execute stage

During the execute stage, the instruction is actually carried out. What happens depends on the type of instruction:

- **Arithmetic/logic operations:** The ALU performs calculations and stores results in the Accumulator
- **Data movement:** Data is transferred between registers or between memory and registers
- **Branch/jump:** The PC is changed to point to a different instruction (breaking the sequence)
- **Input/output:** Data is sent to or received from peripheral devices

After execution completes, the cycle begins again — the next instruction is fetched from the address now in the PC.

1.5 How the cycle repeats

The FDE cycle is **continuous** — as soon as one instruction finishes executing, the cycle begins again with the fetch stage. This repetition happens billions of times per second in modern processors. The only ways the cycle stops are:

- **Program ends:** A HALT instruction tells the CPU to stop
- **System shutdown:** The computer is turned off
- **Error:** An unrecoverable error occurs that stops execution

The speed of the cycle is controlled by the **system clock**, which sends regular pulses to the Control Unit. Each clock pulse triggers the next step in the cycle. A higher clock speed means more cycles per second, which generally means faster processing.

1.6 Trace tables and the FDE cycle

A **trace table** shows how register values change as instructions are executed. This is a useful tool for understanding and debugging programs. During the FDE cycle, you can track how the PC, MAR, MDR, and Accumulator change at each step.

Step	PC	MAR	MDR	ACC	Action
1	100	—	—	0	PC holds address of first instruction
2	100	100	—	0	Address copied from PC to MAR
3	101	100	LOAD 50	0	Instruction fetched to MDR; PC incremented
4	101	100	LOAD 50	0	CU decodes instruction: "Load value 50"
5	101	100	LOAD 50	50	Execute: Value 50 loaded into Accumulator

Trace tables help visualise the flow of data through the CPU during execution. Exam questions may ask you to complete a trace table or describe what happens at specific steps.

1.7 FDE cycle diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│ FETCH-DECODE-EXECUTE CYCLE │
├──────────────────────────────────────────────────────────────────────────┤
│ │ ┌──────────┐ ┌──────────┐ ┌──────────┐ │ │ │ FETCH │ ──────► │ DECODE
──────► │ EXECUTE │ │ │ └──────────┘ └──────────┘ └──────────┘ └──────────┘ │ │ │ │ │ │
│ │ │ └──────────────────────────────────────────────────────────────────────────┘ │ │ (cycle repeats) │ │

├──────────────────────────────────────────────────────────────────────────┤
│ FETCH DECODE EXECUTE │ │ • PC → MAR • MDR → CU • ALU performs │ │ • MAR →
Address bus • CU decodes instruction calculations │ │ • Memory → MDR • CU
prepares signals • Results → ACC │ │ • PC incremented • Or: PC updated │ │
for branch │
└──────────────────────────────────────────────────────────────────────────┘
```

Worked Example 1 — Describing the fetch stage

Question: Describe what happens during the fetch stage of the FDE cycle. [3 marks]

Model answer:
The address of the next instruction is copied from the Program Counter to the Memory Address Register **(1)**.
This address is sent to memory via the address bus **(1)**.
The instruction is retrieved from memory and placed in the Memory Data Register via the data bus **(1)**.

Exam tip: For 3 marks, give three distinct steps. Mentioning "PC increments" can be a valid fourth point if the question allows more marks.

Worked Example 2 — Identifying components in the FDE cycle

Question: State which CPU component is responsible for decoding instructions during the FDE cycle. [1 mark]

Model answer:
The Control Unit (CU) **(1)**

Exam tip: The Control Unit decodes. The ALU executes calculations. Don't confuse them — this is a common error.

Worked Example 3 — Explaining the role of the PC

Question: Explain the role of the Program Counter (PC) in the fetch-decode-execute cycle. [2 marks]

Model answer:
The PC holds the memory address of the next instruction to be fetched **(1)**.
After each fetch, the PC is incremented so it points to the following instruction in sequence **(1)**.

Exam tip: "Role" questions want you to explain what the component does AND how it contributes to the cycle. Give function + behaviour.

Worked Example 4 — Describing register changes during fetch

Question: The Program Counter contains the value 1024. Describe what happens to the MAR and MDR during the fetch stage if the instruction at address 1024 is "ADD 5". [3 marks]

Model answer:
The value 1024 is copied from the PC into the MAR **(1)**.
The MAR sends address 1024 to memory via the address bus **(1)**.
The instruction "ADD 5" is fetched from address 1024 and placed in the MDR **(1)**.

Exam tip: When given specific values, use them in your answer. This shows you can apply your knowledge to a scenario, not just recite definitions.

2. Operating systems

An **operating system (OS)** is system software that manages computer hardware and software resources and provides common services for application programs. Without an OS, users would need to write code to interact directly with hardware — an impractical approach for everyday computing.

2.1 The need for operating systems

Operating systems exist because:

- **Hardware abstraction:** The OS hides the complexity of hardware, providing a consistent interface for applications
- **Resource management:** Multiple programs compete for CPU time, memory, and storage — the OS allocates resources fairly
- **Security:** The OS controls access to files, devices, and system resources
- **User interface:** The OS provides ways to interact with the computer (command line or graphical)

Examples of operating systems include Windows, macOS, Linux, Android, and iOS. Each provides the same core functions but with different interfaces and capabilities.

2.2 File management

The OS organises how files are stored and accessed on secondary storage devices:

- **Directory structure:** Files are organised in a hierarchical tree of folders (directories)
- **File operations:** Creating, reading, writing, copying, moving, deleting files
- **Access control:** Setting permissions (read, write, execute) for different users
- **File location tracking:** The OS maintains records of where file data is physically stored on the disk

2024 exam context: The 2024 Paper 1 asked students to describe file systems using "hierarchical/tree structure" and "root node/directory". Use these terms in your answers.

2.3 Process management

The OS manages running programs (processes) and allocates CPU time:

- **Multitasking:** Running multiple programs apparently simultaneously by rapidly switching between them
- **Process scheduling:** Deciding which process gets CPU time and for how long
- **Process priorities:** Some processes may be given more CPU time than others
- **Process lifecycle:** Starting, pausing, resuming, and terminating processes

When you have multiple applications open (browser, word processor, music player), the OS rapidly switches between them so each appears to run continuously.

2.4 Peripheral management

The OS controls communication with peripheral devices (printers, keyboards, mice, USB devices):

- **Device drivers:** Software that translates OS commands into device-specific instructions
- **Device recognition:** Detecting when devices are connected or disconnected
- **Data buffering:** Temporarily storing data being sent to slower devices (e.g., print queues)
- **Input/output coordination:** Managing the flow of data between devices and the CPU

2.5 User management

The OS manages user accounts and access rights:

- **Authentication:** Verifying user identity through passwords, PINs, or biometrics
- **User profiles:** Storing individual settings, preferences, and files for each user
- **Access control:** Different users have different permissions (admin vs standard user)
- **Activity logging:** Recording user actions for security and accountability

2.6 How OS functions work together

The four OS functions do not operate in isolation — they work together to provide a seamless computing experience. Consider what happens when you open a document:

1. **User management:** Verifies you have permission to access the file
2. **File management:** Locates the file in the directory structure and retrieves it from storage
3. **Process management:** Allocates CPU time to the word processor application
4. **Peripheral management:** Sends the document content to the display for you to see

This integration is why operating systems are essential — they coordinate all these complex operations transparently so users can focus on their tasks rather than managing hardware directly.

2.7 Types of operating system

Different types of operating systems are designed for different purposes:

Desktop/laptop OS

Designed for personal computers with graphical user interfaces.
Examples: Windows, macOS, Linux
Features: Multitasking, networking, user-friendly GUI

Mobile OS

Designed for smartphones and tablets with touch interfaces.
Examples: Android, iOS
Features: Power efficiency, touch input, app stores

While the exam focuses on general OS functions rather than specific operating systems, understanding that different OS types exist helps contextualise the concepts.

Worked Example 5 — Describing file management

Question: Describe two functions of file management performed by an operating system. [4 marks]

Model answer:
Function 1 — Directory structure:
The OS uses a hierarchical/tree structure with a root directory **(1)**.
Each position in the tree is a directory/folder or file, and each file has a unique path relative to the root **(1)**.
Function 2 — Access control:
The OS controls which users can access which files **(1)**.
Permissions can be set to allow or deny read, write, or execute access **(1)**.

Exam tip: The 2024 mark scheme awarded marks for "hierarchical/tree structure", "root node/directory", and "unique path". Use this exact terminology.

Worked Example 6 — Explaining process management

Question: Explain how an operating system allows a user to run multiple applications at the same time. [2 marks]

Model answer:
The OS uses multitasking/time-sharing to rapidly switch between processes **(1)**.
Each process is allocated a small amount of CPU time in turn, so all appear to run simultaneously **(1)**.

Exam tip: "Explain how" requires you to describe the mechanism. Don't just say "the OS manages processes" — explain the switching/scheduling approach.

Worked Example 7 — Describing peripheral management

Question: Describe the role of device drivers in peripheral management. [2 marks]

Model answer:
A device driver is software that allows the OS to communicate with a specific hardware device **(1)**.
It translates general OS commands into device-specific instructions that the peripheral can understand **(1)**.

Exam tip: Device drivers act as "translators" between the OS and hardware. Without them, the OS wouldn't know how to control each unique device.

Worked Example 8 — Comparing OS functions in a scenario

Question: A school network allows teachers and students to log in to any computer. Teachers can access all files, but students can only access their own work. Identify which OS function controls this and explain how it works. [3 marks]

Model answer:
This is controlled by user management **(1)**.
The OS authenticates users when they log in to verify their identity **(1)**.
Different access rights are assigned to different user groups (teachers vs students), controlling which files each can access **(1)**.

Exam tip: Scenario questions want you to apply knowledge to a context. Identify the function first, then explain how it applies to the specific situation described.

3. Utility software and robust software

Utility software performs maintenance tasks that keep computer systems running efficiently and securely. **Robust software** refers to software development practices that ensure reliability, security, and accountability. Both are examined in Paper 1 Topic 3.

3.1 Types of utility software

File repair utilities

File repair utilities detect and fix errors in file systems. They can recover corrupted files and repair damaged directory structures. Operating systems often include built-in file repair tools (like Windows' CHKDSK).

Backup utilities

Backup software creates copies of important data to protect against loss from hardware failure, accidental deletion, or malware. Backups can be:

- **Full backup:** Copies all selected files
- **Incremental backup:** Copies only files changed since the last backup

Compression utilities

Compression software reduces file sizes by encoding data more efficiently. Benefits include saving storage space and reducing transmission time. Common formats include ZIP and RAR.

Defragmentation utilities

When files are saved, they may be split across non-contiguous sectors on a hard disk. Defragmentation reorganises files so their parts are stored together, improving read/write speed. Note: SSDs do not need defragmentation.

Anti-malware utilities

Anti-malware software protects against malicious software:

- **Scanning:** Comparing files against a database of known malware signatures
- **Detection:** Identifying suspicious behaviour or code patterns
- **Quarantine:** Isolating infected files to prevent spread
- **Removal:** Deleting or cleaning malicious code

The 2024 exam included a flowchart question about how anti-virus software works. The process follows a clear sequence: load virus signature database → scan target file → decide if virus detected → quarantine/delete if found, or take no action if clean.

Anti-malware flowchart structure (2024 exam style)

The flowchart for anti-virus software typically follows this sequence:

1. **Start**
2. **Process:** Load virus signature file/database
3. **Process:** Scan/compare target file for virus signature
4. **Decision:** Has a virus been detected? (Yes/No branches)
5. **If Yes:** Quarantine/isolate/fix/delete file
6. **If No:** No further action required
7. **End**

Key exam points: All decision boxes must have exactly two labelled outputs (Yes/No), all arrows must be directed, and there should be no hanging symbols.

When to use each utility

Scenario	Utility needed	Reason
Computer running slowly, HDD	Defragmentation	Fragmented files cause slow read times
Important project files	Backup	Protect against data loss
Sending large files by email	Compression	Reduce file size for faster transmission
Suspicious email attachment opened	Anti-malware	Scan for and remove potential threats
Computer won't boot properly	File repair	Fix corrupted system files

3.2 Robust software practices

Robust software is reliable, secure, and maintainable. The Edexcel specification covers three key practices:

Audit trails

An **audit trail** is a chronological record of system activities that tracks:

- **Who** performed an action (user identification)
- **What** was changed (the specific data or settings modified)
- **When** the change occurred (timestamp)

Benefits of audit trails:

- Improves **accountability** because actions can be traced to individuals

- Helps **detect unauthorised access** or suspicious activity
- Supports **recovery** by showing what was changed and when
- Enables **reverting** to earlier versions if needed

> **2024 exam context:** The 2024 Paper 1 asked "Explain one benefit of an audit trail" for 2 marks. The mark scheme required a linked explanation showing how tracking changes leads to improved accountability or easier recovery.

Code reviews

A **code review** is a systematic examination of source code by developers other than the original author. The purpose is to:

- Identify **bugs and errors** before deployment
- Find **security vulnerabilities** that could be exploited
- Ensure code meets **quality standards** and best practices
- Share knowledge among the development team

Vulnerability identification

Vulnerability identification is the process of finding security weaknesses in software or systems. Methods include:

- **Penetration testing:** Authorised simulated attacks to find weaknesses
- **Code analysis:** Automated tools that scan for common security flaws
- **Security audits:** Systematic reviews of system configurations

Penetration testing (also called "pen testing" or "ethical hacking") involves security professionals attempting to breach a system with the organisation's permission. This helps identify weaknesses before malicious attackers can exploit them. The 2024 exam asked students to explain what penetration testing is and why organisations use it.

Why robust software matters

In professional software development, these practices are not optional — they are essential for:

- **Compliance:** Meeting legal requirements such as GDPR for data protection
- **Reputation:** Security breaches damage customer trust
- **Financial protection:** Data breaches can result in significant fines and losses
- **Reliability:** Well-reviewed code has fewer bugs in production

3.4 Linking audit trails to recovery

A key benefit of audit trails that the exam frequently tests is the ability to support recovery and reverting to earlier versions. Because audit trails record every change:

- Administrators can identify exactly when a problem was introduced
- Changes can be undone by restoring to a point before the error occurred
- The cause of issues can be traced back to specific actions or users
- Patterns of suspicious activity can be identified over time

This is particularly important in database systems where accidental deletions or modifications could have serious consequences. An audit trail provides a safety net that makes recovery possible.

3.5 Compilers and translation

Although primarily a programming concept, the specification includes understanding of how high-level code is translated. A **compiler** translates an entire program from high-level language into machine code before execution.

Key characteristics of a compiler:

- Translates the **whole program in one go**
- Errors are reported **at the end of translation**
- The executable/machine code produced is **architecture-specific** (runs only on the target processor type)
- Once compiled, the program runs **faster** than interpreted code

The architecture-specific nature of compiled code means that a program compiled for Windows on an Intel processor will not run directly on a Mac with an ARM processor. The source code would need to be recompiled for the different architecture.

The 2024 exam asked for two characteristics of a compiler. The mark scheme accepted any combination of: translates whole program at once, errors reported at end, and architecture-specific executable. These are the three key points to remember.

3.6 Comparing compilers and interpreters

While the specification focuses on compilers, understanding the difference from interpreters provides useful context:

Feature	Compiler	Interpreter
Translation	Whole program at once	One line at a time
Error reporting	All errors at end	Stops at first error
Output	Executable file	No separate file
Execution speed	Faster (pre-translated)	Slower (translates during run)
Development	Must recompile after changes	Changes run immediately

Python, which you use for Paper 2, is typically interpreted — this is why you can run your code immediately without a separate compilation step. Languages like C++ are compiled, producing standalone executable files.

Worked Example 9 — Explaining audit trail benefits

Question: Explain one benefit of using an audit trail in a database system. [2 marks]

Model answer:
An audit trail improves accountability **(1)**.
This is because it keeps track of who made changes and when changes were made, so actions can be traced back to individuals **(1)**.

Exam tip: "Explain one benefit" requires a linked pair: state the benefit AND explain why/how it occurs. The 2024 mark scheme required "the expansion must follow/associate with the statement" for full marks.

Worked Example 10 — Describing compiler characteristics

Question: Describe two characteristics of a compiler. [2 marks]

Model answer:
A compiler translates the whole program in one go **(1)**.
Errors are reported at the end of translation / The executable produced is architecture-specific **(1)**.

Exam tip: This exact question appeared in the 2024 Paper 1. The mark scheme accepted any two from: "whole program", "errors at end", "architecture-specific executable".

Worked Example 11 — Explaining code review purpose

Question: Explain how code reviews contribute to robust software development. [3 marks]

Model answer:
Code reviews involve other developers examining source code before deployment **(1)**.
This helps identify bugs and errors that the original programmer may have missed **(1)**.
Security vulnerabilities can be found and fixed before the software is released, reducing the risk of exploitation **(1)**.

Exam tip: "Contribute to robust software" wants you to link code reviews to improved reliability/security. Show the cause-and-effect relationship.

Worked Example 12 — Comparing utility software purposes

Question: A company stores important financial records on a computer system. Explain why both backup software and anti-malware software are essential utilities for this system. [4 marks]

Model answer:
Backup software:
Creates copies of the financial records on separate storage **(1)**.
If the original data is lost or corrupted, it can be restored from the backup, ensuring business continuity **(1)**.
Anti-malware software:
Scans for and detects malicious software such as ransomware **(1)**.

This protects the financial records from being encrypted, stolen, or deleted by malware attacks **(1)**.

Exam tip: For comparison questions, structure your answer clearly with separate paragraphs for each item. Link each utility to the specific context given (financial records).

Worked examples summary

The table below summarises the 12 worked examples covered in this topic, organised by section and difficulty level.

#	Topic	Question type	Marks	Key skill
1	FDE cycle	Describe fetch stage	3	Sequence: PC → MAR → memory → MDR
2	FDE cycle	Identify decode component	1	CU decodes instructions
3	FDE cycle	Explain PC role	2	Function + increment behaviour
4	FDE cycle	Describe registers with values	3	Apply knowledge to specific scenario
5	Operating systems	Describe file management	4	Hierarchical structure + access control
6	Operating systems	Explain process management	2	Multitasking/time-sharing mechanism
7	Operating systems	Describe device drivers	2	Translation between OS and hardware
8	Operating systems	Apply OS functions to scenario	3	Identify function + explain application
9	Robust software	Explain audit trail benefit	2	Linked explanation (2024 style)
10	Compilers	Describe compiler characteristics	2	2024 exam question (exact)
11	Robust software	Explain code review contribution	3	Link to reliability and security
12	Utility software	Compare utilities in context	4	Apply both utilities to scenario

Quick reference

Key terms and definitions

Term	Definition (mark-scheme style)
Fetch-decode-execute cycle	The continuous process by which the CPU fetches instructions from memory, decodes them, and executes them
Fetch stage	The stage where the next instruction is retrieved from memory; PC → MAR, instruction → MDR
Decode stage	The stage where the Control Unit interprets the instruction to determine what operation is required
Execute stage	The stage where the instruction is carried out; may involve ALU calculations or data movement
Operating system	System software that manages computer hardware and software resources and provides services for applications
File management	OS function that organises files in a hierarchical structure, controls access, and tracks file locations
Process management	OS function that allocates CPU time to processes, enabling multitasking
Peripheral management	OS function that controls communication with devices using device drivers
User management	OS function that controls user accounts, authentication, and access rights
Utility software	Software designed to help maintain and optimise computer systems
Defragmentation	Reorganising fragmented files on a hard disk so they are stored contiguously
Audit trail	A chronological record tracking who made changes, what was changed, and when
Code review	Systematic examination of source code by other developers to identify bugs and vulnerabilities
Compiler	Translator that converts an entire high-level program into machine code before execution

FDE cycle register sequence

Step	Action	From	To
1	Copy address of next instruction	PC	MAR
2	Send address to memory	MAR	Address bus → RAM
3	Fetch instruction from memory	RAM	Data bus → MDR
4	Increment program counter	PC	PC + 1
5	Send instruction for decoding	MDR	CU
6	Execute (if calculation)	ALU	Accumulator

Utility software quick reference

Utility	Problem it solves	Key exam phrase
File repair	Corrupted files/file system errors	"Detects and fixes errors in file systems"
Backup	Data loss from failure/attack	"Creates copies to allow restoration"
Compression	Large file sizes	"Reduces file size to save space or speed transmission"
Defragmentation	Slow HDD access (fragmented files)	"Reorganises files to be stored contiguously"
Anti-malware	Viruses, ransomware, etc.	"Scans, detects, quarantines, removes malicious software"

Typical pitfalls

Pitfall 1 — Confusing decode and execute.
Wrong: "The ALU decodes the instruction."
Right: The **Control Unit** decodes instructions. The **ALU** executes arithmetic/logic operations during the execute stage.

Pitfall 2 — Wrong register sequence in fetch.
Wrong: "The instruction is fetched from the MAR."
Right: The MAR holds the **address**. The instruction is fetched from **memory** and placed in the **MDR**.

Pitfall 3 — Forgetting to mention PC increment.
Wrong: Only describing PC → MAR without mentioning PC increment.

Right: After the fetch, the PC is **incremented** to point to the next instruction. This is a valid mark point.

Pitfall 4 — Vague OS function descriptions.
Wrong: "File management manages files."
Right: "File management uses a **hierarchical/tree structure** with a **root directory**, controls **access permissions**, and tracks **file locations**."

Pitfall 5 — Confusing multitasking mechanism.
Wrong: "The OS runs all programs at exactly the same time."
Right: The OS **rapidly switches** between processes, giving each a time slice. They **appear** to run simultaneously.

Pitfall 6 — Defrag on SSDs.
Wrong: "Defragmentation improves SSD performance."
Right: Defragmentation is for **hard disk drives (HDDs)** only. SSDs have no moving parts and don't benefit from defragmentation — it may actually reduce their lifespan.

Pitfall 7 — Missing the "linked" part of audit trail answers.
Wrong: "Audit trails improve accountability." (no explanation)
Right: "Audit trails improve accountability **because** they track who made changes and when, allowing actions to be traced to individuals."

Pitfall 8 — Incomplete compiler characteristics.
Wrong: "A compiler translates code."
Right: "A compiler translates the **whole program in one go**, errors are reported **at the end**, and the executable is **architecture-specific**."

Pitfall 9 — Confusing backup types.
Wrong: "Incremental backup copies all files."
Right: Full backup copies all files. **Incremental backup** copies only files changed since the last backup.

Pitfall 10 — Generic anti-malware descriptions.
Wrong: "Anti-malware removes viruses."
Right: Use the four-step process: "**Scan** files against signature database, **detect** suspicious patterns, **quarantine** infected files, **remove** malware."

Assessment Hub
Long-Answer
Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.